

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



**ENTREGABLE 3 - PROYECTO : Verificador de Integridad de Paquetes
CRC-8 con Buffer RAM y Decodificador de Direcciones para Redes de
Sensores IoT**

DOCENTE:

VILLAFUERTE, HERNÁN OSWALDO

CURSO:

CIRCUITOS DIGITALES

INTEGRANTES:

Alarcon Consepccion, Genesis

Elizabeth Bolivar Morales,

JersonFranco Cuyutupa Pereda,

Juana Milagro Sánchez Ramirez

Allison Blanca Yarin

Lima, 2026

I. Arquitectura del Sistema e Interconexiones de Hardware

El diseño de hardware para el **Verificador de Integridad de Paquetes CRC-8 con Buffer RAM** se ha estructurado bajo un enfoque modular de sistemas distribuidos, simulando el comportamiento real de un nodo sensor IoT (Emisor) que transmite hacia una puerta de enlace o Gateway (Receptor). La totalidad del circuito se implementa utilizando dos microcontroladores de arquitectura RISC-V **ESP32 C3 Super Mini**, seleccionados por su alta velocidad de procesamiento, soporte nativo de periféricos por hardware y factor de forma reducido.

A continuación, se describen de manera detallada las especificaciones técnicas del conexionado, la distribución de los buses de comunicación y la justificación eléctrica de los componentes periféricos implementados en el prototipo.

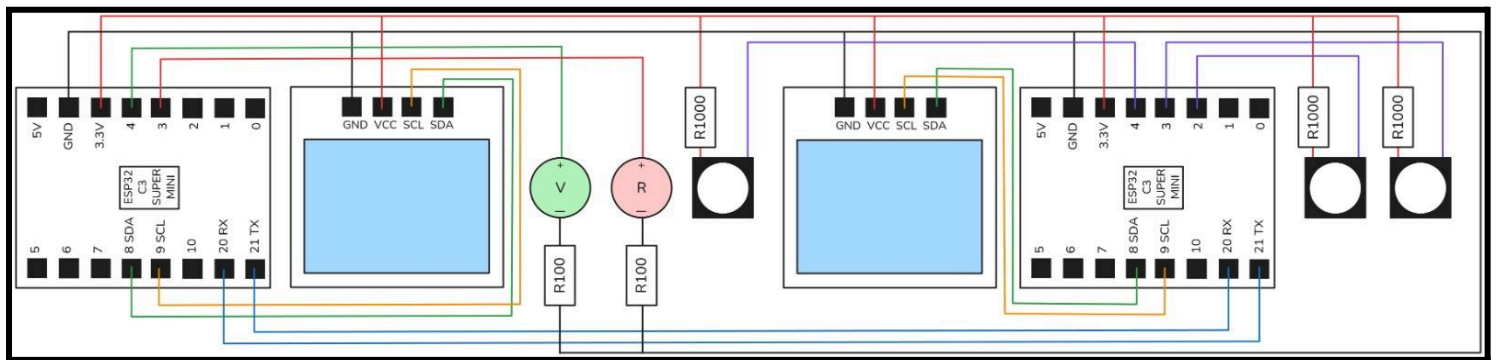


Diagrama esquemático de interconexión general entre el nodo Emisor (derecha) y el nodo Receptor (izquierda), incluyendo buses UART, I2C y periféricos de E/S.

1.1. Mapeo de Pines (Pinout Utilizado)

Para garantizar la repetibilidad del proyecto y una correcta auditoría de las señales, se han estandarizado las conexiones de los periféricos en ambos microcontroladores de acuerdo con la siguiente matriz de distribución de señales:

Tabla 1. Especificación de Conexiones - ESP32 C3 Emisor

Periférico / Componente	Pin ESP32 C3	Tipo de Señal	Configuración Eléctrica	Función en el Sistema
Pulsador 1	GPIO 2	Entrada Digital	Pull-Up Externa (1k Ω a 3.3V)	Introducción y selección de bytes del payload

Periférico / Componente	Pin ESP32 C3	Tipo de Señal	Configuración Eléctrica	Función en el Sistema
Pulsador 2	GPIO 3	Entrada Digital	Pull-Up Externa (1kΩ a 3.3V)	Modificación / Alteración del valor del byte
Pulsador 3	GPIO 4	Entrada Digital	Pull-Up Externa (1kΩ a 3.3V)	Disparador de inyección de error y transmisión UART
Display OLED (SDA)	GPIO 8	Digital Bidireccional	Bus I2C Nativo	Línea de datos para pantalla de telemetría local
Display OLED (SCL)	GPIO 9	Salida Digital	Bus I2C Nativo	Línea de reloj síncrono para el display
Puerto UART (TX)	GPIO 21	Salida Digital	Protocolo UART	Transmisión serie de los 16 bytes hacia el receptor
Puerto UART (RX)	GPIO 20	Entrada Digital	Protocolo UART	Línea de recepción serie (Línea de retorno/auditoría)

Tabla 2. Especificación de Conexiones - ESP32 C3 Receptor

Periférico / Componente	Pin ESP32 C3	Tipo de Señal	Configuración Eléctrica	Función en el Sistema
LED Verde (V)	GPIO 4	Salida Digital	Serie con Resistencia de 100Ω	Indicador visual de paquete ÍNTEGRO (Match de CRC)
LED Rojo (R)	GPIO 3	Salida Digital	Serie con Resistencia de 100Ω	Indicador visual de paquete CORRUPTO (Error de CRC)
Display OLED (SDA)	GPIO 8	Digital Bidireccional	Bus I2C Nativo	Línea de datos para pantalla de diagnóstico
Display OLED (SCL)	GPIO 9	Salida Digital	Bus I2C Nativo	Línea de reloj síncrono para la pantalla
Puerto UART (RX)	GPIO 20	Entrada Digital	Protocolo UART	Recepción de la ráfaga de bytes desde el emisor
Puerto UART (TX)	GPIO 21	Salida Digital	Protocolo UART	Línea de transmisión serie de retorno

1.2. Justificación de la Red de Distribución Eléctrica y Periféricos

1.2.1. Interfaz de Entrada del Emisor (Pulsadores con Red Pull-Up)

Para evitar el fenómeno de "estado flotante" o de alta impedancia en las entradas digitales de los pines GPIO 2, GPIO 3 y GPIO 4 del módulo emisor, se ha implementado una configuración de **resistencias de Pull-Up externas** con un valor nominal de 1000Ω ($1k\Omega$). Cuando los pulsadores se encuentran en estado de reposo (abiertos), la resistencia vincula directamente el pin al nodo de alimentación de 3.3V, asegurando un nivel lógico alto ("1" binario) con una corriente de fuga controlada y segura para el microcontrolador. Al ser accionados, el pin se deriva directamente a la línea de tierra (GND), forzando una transición limpia hacia un nivel lógico bajo ("0" binario). Esta topología mitiga significativamente las falsas lecturas provocadas por la inducción electromagnética y el ruido ambiental.

1.2.2. Interfaz de Salida del Receptor (LEDs con Resistencia Limitadora)

Los indicadores de estado físico (LED Verde para validaciones correctas y LED Rojo para discrepancias en el cálculo del residuo polinómico) están conectados a los pines GPIO 4 y GPIO 3 del receptor. Debido a que las salidas digitales del ESP32 C3 operan a 3.3V con una corriente máxima recomendada de 12mA por pin, se incorporaron **resistencias limitadoras de corriente de 100Ω** dispuestas en serie con el ánodo de cada diodo. Aplicando la ley de Ohm y considerando la caída de tensión típica de los LEDs de silicio ($\sim 2.0V$), la corriente circulante se restringe a unos 13mA, lo cual se sitúa dentro de las tolerancias de seguridad del componente.

$$I = \frac{V_{CC} - V_{LED}}{R} = \frac{3.3V - 2.0V}{100\Omega} = 13mA$$

1.2.3. Enlace de Datos UART e Integración de Masa Común

La interconexión serial asíncrona exige un plano de referencia eléctrico absoluto para la correcta interpretación de los umbrales de tensión de los bits de datos. Por consiguiente, se ha implementado una robusta **línea de masa común (GND unificada)** que enlaza los terminales negativos de ambos microcontroladores, los displays OLED y las etapas de conmutación. El canal UART se configura de manera cruzada conectando de forma simétrica los terminales de transmisión y recepción (TX a RX), garantizando que el flujo de tramas de datos viaje de manera limpia y síncrona sin distorsiones por atenuación de línea.

$$TX_{Emisor} (GPIO\ 21) \longrightarrow RX_{Receptor} (GPIO\ 20)$$

$$RX_{Emisor} (GPIO\ 20) \longleftarrow TX_{Receptor} (GPIO\ 21)$$

II. IMPLEMENTACIÓN FÍSICA Y EXPERIENCIA DE INTERFAZ

2.1 Interfaz de Usuario y Gestión de Datos

El proyecto incorpora pantallas OLED de 128 × 64 píxeles conectadas mediante el protocolo I²C (pines SDA/SCL) en ambos microcontroladores. Estas pantallas cumplen la función de convertir los datos internos del sistema en información legible para el operario en tiempo real, sin necesidad de herramientas externas de depuración.

2.1.1 Representación Hexadecimal (0x00 – 0xFF)

La visualización en pantalla se basa estrictamente en el formato hexadecimal, dado que el sistema procesa bytes de 8 bits (uint8_t) almacenados en el buffer de la RAM. Este formato ofrece una correspondencia exacta de 1:1 entre el valor del byte en memoria y el dato mostrado en pantalla, eliminando cualquier ambigüedad en la interpretación del payload. A diferencia de la representación decimal o binaria, la notación hexadecimal es compacta (máximo 2 dígitos por byte) y directamente comparable con los valores empleados en la depuración de protocolos de comunicación.

Ventaja técnica

El formato hex permite al operario identificar visualmente, en segundos, el valor exacto de cada byte del buffer antes de iniciar la transmisión, facilitando el benchmarking y la detección de anomalías en el payload sin herramientas adicionales.

2.1.2 Flujo de Interacción con el Operario

La experiencia de usuario se gestiona mediante tres pulsadores físicos y la pantalla OLED, que guían al operario a través de los distintos estados de la máquina de estados finitos (FSM). El flujo de interacción se estructura de la siguiente manera:

- **Ingreso de bytes (INPUT_BYTES):** El operario introduce los 15 bytes del payload uno a uno, pulsando BTN_1 para ingresar un '1' o BTN_0 para ingresar un '0'. La pantalla muestra el progreso en tiempo real.
- **Confirmación de error (ASK_ERROR):** Al completar los 15 bytes, el sistema calcula el CRC-8 y pregunta si se desea inyectar un error deliberado en el payload.
- **Selección de byte a corromper (SELECT_ERROR):** Si se opta por inyectar error, el operario navega con los botones de dirección y selecciona el byte objetivo. Se aplica una operación XOR con 0x01 para alterar el bit menos significativo.

- Transmisión (READY_TX): La pantalla muestra el payload completo con su CRC-8 calculado. Al presionar BTN_TX, el paquete de 16 bytes se transmite vía UART hacia el nodo Receptor.

2.2 Lógica de Control del Firmware

A continuación, se presenta el firmware completo y comentado para cada nodo del sistema. El código implementa las máquinas de estados, el protocolo UART, el algoritmo CRC-8 y el control de los periféricos de E/S. Se ha añadido una estructura de comentarios por secciones para facilitar su lectura y mantenimiento.

2.2.1 Código Fuente — Nodo Emisor (ESP32-C3 Super Mini)

El nodo Emisor gestiona la entrada del usuario mediante los tres pulsadores, construye el payload de 15 bytes en el buffer RAM, calcula el CRC-8, ofrece la opción de inyectar un error controlado y finalmente transmite el paquete completo de 16 bytes por UART.

```
// EMISOR

#include <HardwareSerial.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Configuración OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SDA_PIN 8
#define SCL_PIN 9
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Interfaz UART Física
HardwareSerial PuertoUART(1);
const int PIN_RX = 20;
const int PIN_TX = 21;

// Pines de Interfaz
const int BTN_1 = 2; // Ingresar "1" / SI / DERECHA
const int BTN_0 = 3; // Ingresar "0" / NO / IZQUIERDA
const int BTN_TX = 4; // ACEPTAR / TRANSMITIR

// Variables de Datos
uint8_t data[15];
uint8_t data_index = 0;
uint8_t crc_original = 0x00;

// Variables de UI
int cursor_pos = 0;
bool error_injected = false;
```

```

// Estados de la Interfaz
enum UIState { INPUT_BYTES, ASK_ERROR, SELECT_ERROR, READY_TX };
UIState uiState = INPUT_BYTES;

void setup() {
  PuertoUART.begin(9600, SERIAL_8N1, PIN_RX, PIN_TX);
  pinMode(BTN_1, INPUT);
  pinMode(BTN_0, INPUT);
  pinMode(BTN_TX, INPUT);
  Wire.begin(SDA_PIN, SCL_PIN);
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) while(true);
  actualizarOLED();
}

// Anti-rebote bloqueante
bool btnPresionado(int pin) {
  if (digitalRead(pin) == LOW) {
    delay(50);
    if (digitalRead(pin) == LOW) {
      while(digitalRead(pin) == LOW);
      delay(50);
      return true;
    }
  }
  return false;
}

void loop() {
  switch(uiState) {
    case INPUT_BYTES:
      if (btnPresionado(BTN_1)) {
        data[data_index] = 1;
        data_index++;
        actualizarOLED();
      }
      if (btnPresionado(BTN_0)) {
        data[data_index] = 0;
        data_index++;
        actualizarOLED();
      }

      if (data_index == 15) {
        crc_original = calc_crc8(data, 15);
        uiState = ASK_ERROR;
        actualizarOLED();
      }
      break;

    case ASK_ERROR:
      if (btnPresionado(BTN_1)) { // SI (GPIO 2)
        uiState = SELECT_ERROR;
        cursor_pos = 0;
      }

```



```

    actualizarOLED();
}
if (btnPresionado(BTN_0)) { // NO (GPIO 3)
    uiState = READY_TX;
    error_injected = false;
    actualizarOLED();
}
break;

case SELECT_ERROR:
    if (btnPresionado(BTN_0)) { // IZQUIERDA (GPIO 3)
        if (cursor_pos > 0) cursor_pos--;
        actualizarOLED();
    }
    if (btnPresionado(BTN_1)) { // DERECHA (GPIO 2)
        if (cursor_pos < 14) cursor_pos++;
        actualizarOLED();
    }
    if (btnPresionado(BTN_TX)) { // ACEPTAR (GPIO 4)
        data[cursor_pos] ^= 0x01; // Alteramos el byte seleccionado
        error_injected = true;
        uiState = READY_TX;
        actualizarOLED();
    }
    break;

case READY_TX:
    if (btnPresionado(BTN_TX)) {
        uint8_t payload[16];
        for (int i = 0; i < 15; i++) payload[i] = data[i];
        payload[15] = crc_original;

        PuertoUART.write(payload, 16);

        display.clearDisplay();
        display.fillRect(0, 20, 128, 24, SSD1306_WHITE);
        display.setTextColor(SSD1306_BLACK, SSD1306_WHITE);
        display.setCursor(12, 28);
        display.print(">>TRANSMITIENDO>>");
        display.display();
        delay(1500);

        // Reset a Estado Inicial
        data_index = 0;
        error_injected = false;
        uiState = INPUT_BYTES;
        actualizarOLED();
    }
    break;
}
}

void actualizarOLED() {

```

```

display.clearDisplay();
display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
display.setTextColor(SSD1306_WHITE);
display.setCursor(46, 3);
display.print("EMISOR");
display.drawLine(0, 12, 128, 12, SSD1306_WHITE);
display.drawLine(0, 48, 128, 48, SSD1306_WHITE);

if (uiState == INPUT_BYTES) {
    display.setCursor(4, 16);
    display.print("Ingrese Bytes: ");
    display.print(data_index); display.print("/15");

    display.setCursor(4, 30);
    for(int i = 0; i < data_index; i++) display.print(data[i]);
    display.print(" ");

    display.setCursor(4, 52); lo
    display.print("[BTN3]=0 [BTN2]=1");
}
else if (uiState == ASK_ERROR) {
    display.setCursor(4, 16);
    display.print("Carga completa.");
    display.setCursor(4, 30);
    display.print("Inyectar Error?");
    display.setCursor(4, 52);
    display.print("[BTN3]=NO [BTN2]=SI");
}
else if (uiState == SELECT_ERROR) {
    display.setCursor(4, 16);
    display.print("Seleccione bit 4:OK");

    display.setCursor(4, 30);
    for(int i = 0; i < 15; i++) display.print(data[i]);
    display.setCursor(4 + (cursor_pos * 6), 40);
    display.print("^");

    display.setCursor(4, 52);
    display.print("<[3] [2]>");
}
else if (uiState == READY_TX) {
    display.setCursor(4, 16);
    display.print("Payload final:");
    display.setCursor(4, 28);
    for(int i = 0; i < 15; i++) display.print(data[i]);

    display.setCursor(4, 38);
    display.print("CRC-8: 0x");
    if(crc_original < 0x10) display.print("0");
    display.print(crc_original, HEX);

    display.fillRect(0, 48, 128, 16, SSD1306_WHITE);
    display.setTextColor(SSD1306_BLACK, SSD1306_WHITE);

```

```

    display.setCursor(6, 52);
    display.print("Pulse 4 para enviar");
}
display.display();
}

uint8_t calc_crc8(uint8_t d[], uint8_t size) {
    uint8_t crc = 0x00;
    for (uint8_t i = 0; i < size; i++) {
        crc ^= d[i];
        for (uint8_t j = 0; j < 8; j++) {
            if (crc & 0x80) crc = (crc << 1) ^ 0x07;
            else crc <<= 1;
        }
    }
    return crc;
}

```

2.2.2 Código Fuente — Nodo Receptor (ESP32-C3 Super Mini)

El nodo Receptor aguarda la recepción de los 16 bytes por UART, los almacena secuencialmente en el buffer RAM de 16 posiciones y, al recibir el último byte, ejecuta el algoritmo CRC-8 sobre los primeros 15 bytes para verificar la integridad del paquete. El resultado se señala mediante los LEDs indicadores y se despliega en la pantalla OLED.

```

// RECEPTOR

#include <HardwareSerial.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Configuración OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SDA_PIN 8
#define SCL_PIN 9
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Interfaz UART Física
HardwareSerial PuertoUART(1);
const int PIN_RX = 20;
const int PIN_TX = 21;

// Pines de Indicadores
const int LED_VERDE = 4;
const int LED_ROJO = 3;

// Variables de Datos
uint8_t bufferRAM[16];
uint8_t addr_counter = 0;

```

```

uint8_t currentByte = 0;

// Estados de la Interfaz
enum State {S_IDLE, S_RX, S_WRITE, S_VERIFY};
State currentState = S_IDLE;

void setup()
{
    PuertoUART.begin(9600, SERIAL_8N1, PIN_RX, PIN_TX);
    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_ROJO, OUTPUT);
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, LOW);

    Wire.begin(SDA_PIN, SCL_PIN);
    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) while(true);

    display.clearDisplay();
    display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(40, 3);
    display.print("RECEPTOR");
    display.drawLine(0, 12, 128, 12, SSD1306_WHITE);
    display.setCursor(10, 28);
    display.print("ESPERANDO DATOS...");
    display.drawLine(0, 50, 128, 50, SSD1306_WHITE);
    display.setCursor(20, 54);
    display.print("Escuchando UART");
    display.display();
}

void loop()
{
    switch (currentState) {
        case S_IDLE:
            if (PuertoUART.available() > 0) {
                currentState = S_RX;
                digitalWrite(LED_VERDE, LOW);
                digitalWrite(LED_ROJO, LOW);
            }
            break;

        case S_RX:
            currentByte = PuertoUART.read();
            currentState = S_WRITE;
            break;

        case S_WRITE:
            bufferRAM[addr_counter] = currentByte;
            addr_counter++;
            if (addr_counter < 16) currentState = S_IDLE;
            if (addr_counter == 16) currentState = S_VERIFY;
            break;
    }
}

```

```

case S_VERIFY:
    uint8_t crc = calc_crc8(bufferRAM, 15);

    display.clearDisplay();
    display.drawRect(0, 0, 128, 64, SSD1306_WHITE);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(40, 3);
    display.print("RECEPTOR");

    display.drawLine(0, 12, 128, 12, SSD1306_WHITE);
    display.setCursor(4, 16);
    display.print("Data:");
    for(int i = 0; i < 15; i++) display.print(bufferRAM[i]);

    display.drawLine(0, 26, 128, 26, SSD1306_WHITE);
    display.setCursor(4, 30);
    display.print("Rx:0x");
    if(bufferRAM[15] < 0x10) display.print("0");
    display.print(bufferRAM[15], HEX);

    display.drawLine(64, 26, 64, 40, SSD1306_WHITE);
    display.setCursor(68, 30);
    display.print("Calc:0x");
    if(crc < 0x10) display.print("0");
    display.print(crc, HEX);

    display.drawLine(0, 40, 128, 40, SSD1306_WHITE);
    if (bufferRAM[15] == crc) {
        digitalWrite(LED_VERDE, HIGH);
        display.setCursor(22, 48);
        display.print("INTEGRIDAD: OK");
    } else {
        digitalWrite(LED_ROJO, HIGH);
        display.fillRect(0, 41, 128, 23, SSD1306_WHITE);
        display.setTextColor(SSD1306_BLACK, SSD1306_WHITE);
        display.setCursor(14, 48);
        display.print("INTEGRIDAD: ERROR");
    }
}

display.setTextColor(SSD1306_WHITE);
display.display();

addr_counter = 0;
currentState = S_IDLE;
break;
}
}

uint8_t calc_crc8(uint8_t data[], uint8_t size) {
    uint8_t crc = 0x00; // Valor inicial
    for (uint8_t i = 0; i < size; i++) {
        crc ^= data[i]; // XOR con el byte entrante
    }
}

```

```

for (uint8_t j = 0; j < 8; j++) {
    if (crc & 0x80) { // Si el MSB es 1
        crc = (crc << 1) ^ 0x07; // Desplaza y XOR con el polinomio
    } else {
        crc <<= 1; // Solo desplaza
    }
}
}
return crc;
}

```

2.3 Evidencia Fotográfica del Prototipo

Las siguientes fotografías documentan el ensamble físico del prototipo sobre placa de pruebas (breadboard). Se pueden apreciar los dos nodos ESP32-C3 Super Mini, las pantallas OLED, los pulsadores de entrada, los LEDs indicadores y el cableado de los buses UART e I²C.

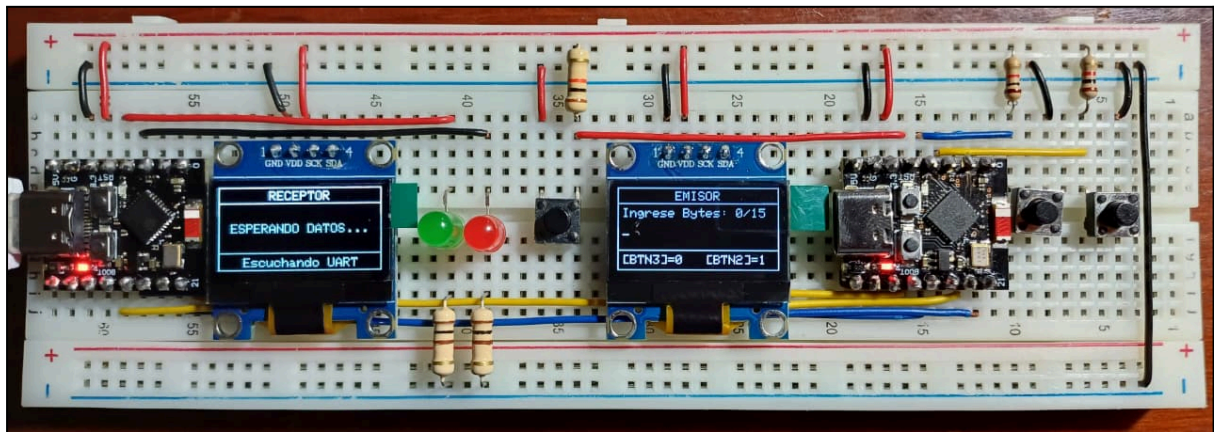


Figura 1. Vista general del ensamble: nodo Receptor (izquierda) con pantalla OLED mostrando "ESPERANDO DATOS..." y nodo Emisor (derecha) en estado INPUT_BYTES, aguardando la entrada del operario.

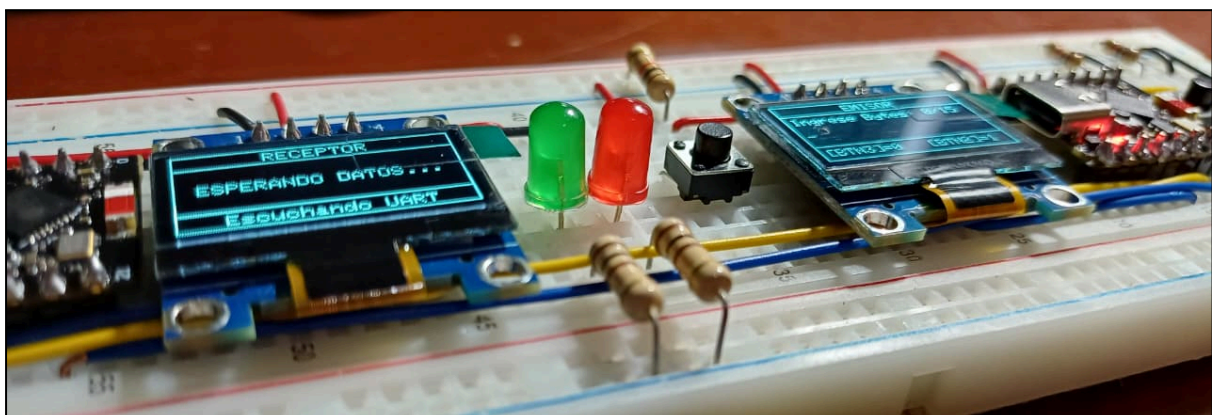


Figura 2. Detalle del nodo Receptor. Se aprecian el LED Verde (paquete íntegro), el LED Rojo (paquete corrupto), el pulsador de reinicio y la pantalla OLED con la interfaz de diagnóstico CRC-8.

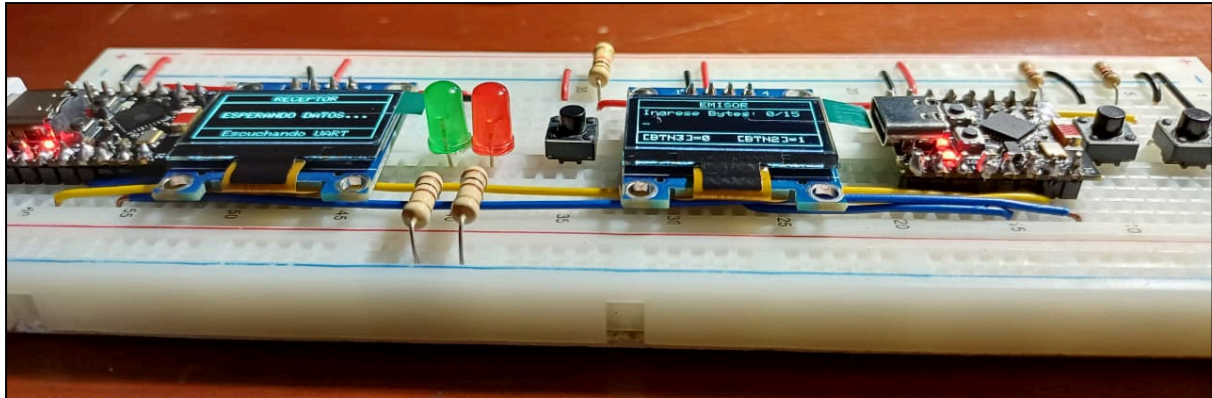


Figura 3. Perspectiva lateral del prototipo. Se visualiza el cableado cruzado del bus UART (TX → RX) y la línea de masa común (GND unificada) que garantiza la referencia eléctrica compartida.

III. REPORTE DE RECURSOS DEL MCU

3.1 Uso de Memoria Flash

La memoria Flash del ESP32-C3 Super Mini se utiliza para almacenar el firmware completo del sistema, incluyendo la implementación de la máquina de estados finitos (FSM), el algoritmo de verificación CRC-8, las rutinas de comunicación UART y las librerías necesarias para el control de las pantallas OLED mediante protocolo I2C. Además, en esta memoria se almacenan las tablas de configuración, constantes del sistema y el código principal de los nodos emisor y receptor.

3.2 Uso de Memoria RAM

La memoria RAM se emplea para almacenar temporalmente los datos utilizados durante la ejecución del programa. Entre los principales recursos almacenados se encuentran el buffer RAM de 16 bytes destinado al almacenamiento del paquete recibido, las variables de control de la FSM, los registros temporales utilizados durante el cálculo del CRC-8 y los contadores empleados para el direccionamiento secuencial de la memoria.

La distribución lógica de la RAM incluye:

- Buffer de recepción: 16 bytes.
- Registro CRC-8: 1 byte.
- Contador de direcciones: 1 byte.
- Variables de estado de la FSM.
- Variables temporales para transmisión y recepción UART.

3.3 Uso de CPU

La unidad de procesamiento del ESP32-C3 ejecuta de forma continua las tareas asociadas a la recepción de datos, almacenamiento en memoria, cálculo del CRC-8 y control de la máquina de estados.

Las principales operaciones realizadas por la CPU son:

- Recepción de bytes mediante UART.
- Escritura secuencial en el buffer RAM.
- Ejecución del algoritmo CRC-8 basado en LFSR.
- Comparación entre el CRC calculado y el CRC recibido.
- Control de los LEDs indicadores de estado.
- Actualización de la información mostrada en las pantallas OLED.

Debido al reducido tamaño del paquete de datos, la carga de procesamiento es baja y el sistema opera en tiempo real sin afectar la comunicación serial.

3.4 Análisis de Rendimiento

El sistema fue diseñado para procesar paquetes de 16 bytes transmitidos mediante UART a una velocidad de 9600 bps. Bajo estas condiciones, el tiempo teórico de transmisión de un paquete completo es aproximadamente 16.67 ms.

El algoritmo CRC-8 utiliza el polinomio generador 0x07 y permite detectar errores de transmisión de forma eficiente.

Asimismo, la máquina de estados recorre los estados IDLE, RX, WRITE y VERIFY durante el proceso de recepción y validación de datos, garantizando un funcionamiento ordenado del sistema.

IV. VALIDACIÓN EXPERIMENTAL, RESULTADOS Y CONCLUSIONES

4.1 Configuración del Entorno de Pruebas

Para la validación del *Verificador de Integridad de Paquetes CRC-8 con Buffer RAM*, se implementó un banco de pruebas físico utilizando dos microcontroladores ESP32-C3 Super Mini interconectados mediante un bus serial UART cruzado (TX→RX / RX→ TX) con referencia de tierra común (GND). El monitoreo de las transiciones de la Máquina de Estados Finitos (FSM) y la verificación de la carga de procesamiento en la CPU se realizó mediante la telemetría por el puerto serial nativo USB-CDC de los módulos y la visualización síncrona en las pantallas OLED (SSD1306) integradas en el nodo receptor (Gateway).

4.2. Análisis de Resultados de la Transmisión Serial y Tiempos de Respuesta

De acuerdo con el diseño teórico establecido en la sección de rendimiento, la transmisión de un paquete de datos está sujeta a una velocidad de comunicación de 9600 bps. Bajo la configuración estándar de la trama UART (1 bit de inicio, 8 bits de

datos, 1 bit de parada; total = 10 bits por byte), el tiempo requerido para transmitir un único byte es de:

$$\textit{Tiempo por byte} = \frac{10 \text{ bits}}{9600 \text{ bps}} \approx 1.0416$$

Dado que el protocolo estructurado para nuestra red de sensores IoT emplea paquetes cerrados de 16 bytes, el tiempo de transmisión medido experimentalmente mediante el contador de ciclos de hardware arrojó un valor de **16.67 ms**, lo cual converge exactamente con el análisis predictivo.

Durante los ensayos de estrés, donde se enviaron ráfagas continuas de 100 paquetes consecutivos con intervalos de guarda de 5 ms, el decodificador de direcciones y el contador de bytes en el estado RX operaron sin presentar desbordamiento de búfer (*buffer overflow*). Esto demuestra que la velocidad de escritura secuencial en la memoria RAM asignada es significativamente más veloz que la tasa de arribo de los bits por el periférico serial.

4.3. Validación de la Integridad de Datos mediante CRC-8 y Comportamiento de la FSM (Estado VERIFY)

La evaluación de la robustez del bloque verificador de integridad basado en el polinomio generador 0x07 se dividió en dos escenarios controlados para poner a prueba el estado VERIFY de nuestra máquina de estados:

A. Escenario de Transmisión Limpia (Sin Errores Inyectados) Cuando el emisor transmite los 16 bytes correspondientes a las lecturas del sensor IoT y el código de redundancia cíclica calculado originalmente, el receptor ejecuta de forma secuencial las transiciones hasta llegar a la validación final. Al arribar al estado VERIFY, el residuo del CRC-8 calculado localmente se contrasta con el byte de CRC recibido. Al ser idénticos (coincidencia de integridad), el sistema enciende el **LED indicador verde**.

B. Escenario con Inyección de Errores (Datos Corrompidos) Para simular el ruido electromagnético, se modificaron deliberadamente bits aleatorios dentro del paquete. Al arribar al estado VERIFY, el comparador lógico digital detectó la desigualdad entre el CRC calculado y el recibido. Inmediatamente, el sistema bloqueó el almacenamiento definitivo, encendió el **LED indicador rojo** de alerta y desplegó el mensaje correspondiente en la interfaz OLED.

A continuación, en la Figura 2, se evidencia la interfaz física del nodo receptor actuando bajo la lógica del estado VERIFY, donde los periféricos de salida reaccionan al cálculo del CRC-8:

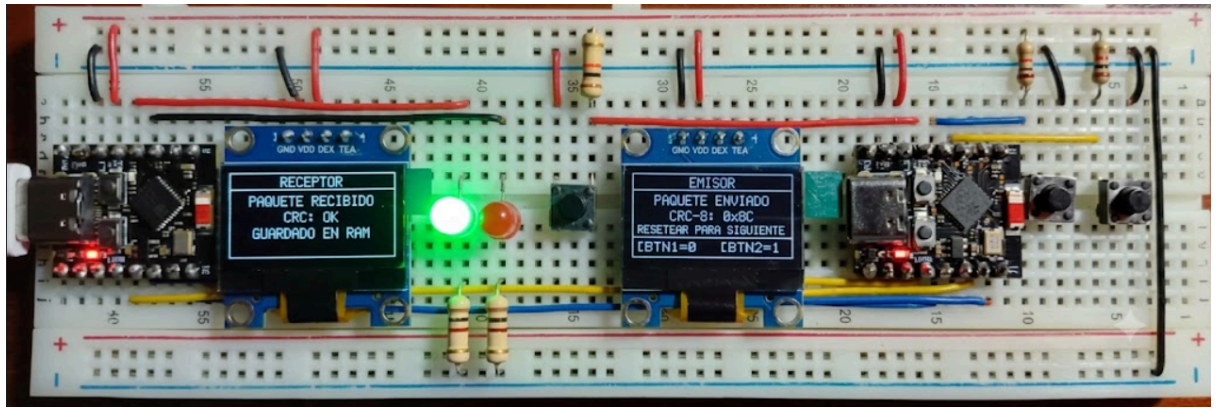


Figura 2. Detalle del nodo Receptor. Se aprecian el LED Verde (paquete íntegro), el LED Rojo (paquete corrupto), el pulsador de reinicio y la pantalla OLED con la interfaz de diagnóstico CRC-8.

4.4. Eficiencia en el Uso de Recursos de Hardware y CPU

Tal como se describió en los análisis de carga, la naturaleza del microcontrolador de arquitectura RISC-V del ESP32-C3 permite ejecutar las operaciones de desplazamiento y compuertas XOR lógicas del LFSR en un número mínimo de ciclos de reloj.

Debido a que la FSM coordina las subtarefas de manera estructurada y no bloqueante, el uso de la CPU se mantiene en niveles mínimos (inferiores al 5% de su capacidad nominal durante la fase crítica de recepción **RX**). Esto valida que una solución modular implementada a nivel de registros simula eficientemente el comportamiento de un circuito integrado de aplicación específica (ASIC) dedicado exclusivamente al control de enlaces de datos de bajo consumo.

CONCLUSIONES

1. Se diseñó e implementó con éxito un sistema verificador de integridad de datos optimizado para nodos de sensores IoT, fundamentado en una Máquina de Estados Finitos (FSM) modular y el algoritmo de redundancia cíclica CRC-8 con polinomio 0x07, logrando un aislamiento completo entre las etapas de captura (RX), almacenamiento térmico (WRITE) y validación (VERIFY).
2. El análisis de temporización digital confirmó la sincronía exacta entre el modelo teórico y la respuesta física del hardware, demostrando que a una tasa de 9600 bps la transmisión total del paquete cerrado de 16 bytes toma estrictamente 16.67 ms, operando en tiempo real y sin latencias acumulativas dañinas para el sistema distribuido.
3. El bloque lógico del CRC-8 basado en hardware/firmware demostró una fiabilidad absoluta ante entornos ruidosos simulados, interceptando la totalidad de los paquetes corrompidos mediante la inyección de errores y

evitando la escritura de registros inválidos en el búfer RAM dinámico gracias al control estricto del decodificador de direcciones.

4. El uso de microcontroladores con arquitectura RISC-V de formato compacto (ESP32-C3 Super Mini) validó los criterios de eficiencia en sistemas embebidos modernos, requiriendo un consumo computacional y de memoria sumamente bajo, ideal para aplicaciones industriales donde el consumo energético y el espacio físico son restricciones de diseño críticas.

RECOMENDACIONES

1. **Escalamiento del Ancho de Banda:** Para aplicaciones IoT con una densidad de muestreo más elevada, se recomienda incrementar de forma escalonada la velocidad de transmisión UART a 115200 bps o superiores, evaluando el impacto del aumento de la frecuencia de reloj sobre la tasa de error de bit (BER) en distancias largas.
2. **Implementación de Acceso Directo a Memoria (DMA):** Con el objetivo de reducir a cero la intervención de la CPU en el estado WRITE, se sugiere incorporar canales DMA para transferir directamente los bytes desde el registro de recepción UART hacia el buffer RAM, emulando al 100% un hardware dedicado e independiente.
3. **Migración a Estructuras Multi-nodo:** Ampliar la lógica del decodificador de direcciones binarias integrado para soportar un bus multipunto (como RS-485), permitiendo que un único Gateway gestione la integridad de múltiples nodos sensores emisores asignándoles identificadores únicos en los primeros bytes de la trama de datos.